1. Make use of package. Write steps to create a Package with example.

A **package** in Java is a group of related classes and interfaces. It helps in **organizing code**, **avoiding name conflicts**, and **providing access control**. Java provides two types of packages: **built-in packages** (like java.util, java.io) and **user-defined packages**.

Steps to Create a Package

1. **Create a folder** where your package will be stored.
2. **Create a Java file** and write the package statement at the top:
3. `package packname;`
4. **Write the class code normally.**
5. **Compile the file** using `javac -d . ClassName.java`
    - `-d` creates the folder structure automatically.
6. **Use the package in another class** using `import packname.ClassName;`

Example

File: Student.java

```
package mypack;

public class Student {
    public void show() {
        System.out.println("Hello from Package");
    }
}
```

Compile it:

```
javac -d . Student.java
```

File: TestPackage.java

```
import mypack.Student;

public class TestPackage {
    public static void main(String[] args) {
        Student s = new Student();
        s.show();
    }
}
```

Run:

```
java TestPackage
This will print:
Hello from Package
```

2. Develop program with different visibility controls used in Java.

In Java, **visibility controls** (access modifiers) define how classes, methods, and variables can be accessed. The four main access modifiers are:

- **public** – accessible from anywhere
- **protected** – accessible within same package + subclasses
- **default (no modifier)** – accessible only within the same package
- **private** – accessible only within the same class
-

Example Program Showing All Visibility Controls

```
public class A {
    public int pub = 10;          // public
    protected int pro = 20;       // protected
    int def = 30;                 // default
    private int pri = 40;         // private

    public void show() {
        System.out.println("Public: " + pub);
        System.out.println("Protected: " + pro);
        System.out.println("Default: " + def);
        System.out.println("Private: " + pri);
    }
}
```

4. List four inbuilt exceptions. Explain any one with example.

Four Inbuilt Exceptions in Java

1. **ArithmeticException**
2. **NullPointerException**
3. **ArrayIndexOutOfBoundsException**
4. **NumberFormatException**

Explain Any One: ArithmeticException

This exception occurs when an illegal arithmetic operation is performed, such as **division by zero**.

Example

```
public class Example {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;   // causes ArithmeticException
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed!");
        }
    }
}
```

Output

```
Error: Division by zero is not allowed!
```

5. Explain Thread Lifecycle.

A **thread lifecycle** represents the different states a thread goes through from creation to termination. Java threads have **five main states**:

1. New (Created State)

- Thread object is created using the `Thread` class.
- Thread is not yet started.

2. Runnable (Ready-to-Run State)

- Thread becomes runnable after calling `start()`.
- It is ready to run but waiting for CPU scheduling.

3. Running (Executing State)

- Thread is actually executing its `run()` method.
- JVM scheduler picks the thread and gives CPU time.

4. Blocked / Waiting / Sleeping

Thread temporarily stops running due to:

- `sleep()` method
- waiting for I/O
- waiting for lock
- waiting for another thread to complete (`join()`)

5. Terminated (Dead State)

- Thread ends after completing `run()` method.
- It cannot be restarted.

Diagram (Simple)

New → Runnable → Running → (Waiting/Sleeping/Blocked) → Runnable → Running → Terminated

---

7. Explain User Defined Exception.

A **User Defined Exception** (custom exception) is an exception created by the programmer. It is useful when Java's built-in exceptions are not enough and you want to handle **specific application errors**.

To create a custom exception:

1. Create a class that **extends `Exception`** (or `RuntimeException`).
2. Provide a constructor to pass a message.
3. Use `throw` to raise the exception.

Short Example

```
class AgeException extends Exception {
    AgeException(String msg) {
        super(msg);
```

```
    }
}

public class Test {
    public static void main(String[] args) {
        try {
            int age = 15;
            if (age < 18)
                throw new AgeException("Age below 18 not allowed");
        }
        catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

```
Age below 18 not allowed
```

---

10. Program: One thread prints even numbers, another prints odd numbers between 1 to 10

Here is a clean **medium-length** and **simple** answer:

Java Program

```
class EvenThread extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0)
                System.out.println("Even: " + i);
        }
    }
}

class OddThread extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 != 0)
                System.out.println("Odd: " + i);
        }
    }
}

public class NumberDemo {
    public static void main(String[] args) {
        new EvenThread().start();
        new OddThread().start();
    }
```

```
}
```

12. Develop a Java program to handle 'Divide by Zero' / Arithmetic Exception

Program

```
public class DivideDemo {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int c = a / b;      // causes ArithmeticException
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
        }
    }
}
```

Output

```
Error: Cannot divide by zero!
```

---

13. Define stream classes in Java. Explain InputStream class.

Stream Classes in Java

In Java, **streams** are used to perform **input and output (I/O)** operations.
They handle data in the form of **bytes** or **characters**.

Java provides two main types of streams:

1. **Byte Streams**
    - Read/write data in **bytes** (8-bit).
    - Classes: `InputStream`, `OutputStream`
2. **Character Streams**
    - Read/write data in **characters** (16-bit Unicode).
    - Classes: `Reader`, `Writer`

Other stream categories:

- **File Streams**
- **Buffered Streams**
- **Data Streams**

InputStream Class

`InputStream` is an **abstract class** used to read data **byte by byte** from a source.

Key Points

- It is the **superclass** for all byte-input streams.
- Reads raw bytes from files, memory, network, etc.
- Common subclasses:
  - **FileInputStream**
  - **BufferedInputStream**
  - **DataInputStream**
  - **ByteArrayInputStream**

Important Methods

- `int read()` → reads one byte
- `int read(byte[] b)` → reads bytes into an array
- `void close()` → closes the stream

---

15. Explain InputStreamReader and OutputStreamWriter (Short & Medium)

InputStreamReader

- It is a **bridge** between **byte streams** and **character streams**.
- Converts **bytes → characters**.
- Commonly used when reading text data from byte-based sources like files or network streams.

Example:

```
InputStreamReader in = new InputStreamReader(System.in);
int ch = in.read();
System.out.println((char) ch);
```

OutputStreamWriter

- Converts **characters → bytes**.

- Used when writing text data to byte-based outputs.

Example:

```
OutputStreamWriter out = new OutputStreamWriter(System.out);
out.write("Hello");
out.flush();
```

---

17. Explain following methods: read(), write(), skip(), close() (Short & Medium)

1. read()

- Used to **read data** (byte or character) from a stream.
- Returns **-1** when end of file is reached.

Example:

```
int x = in.read();
```

2. write()

- Used to **write data** (byte or character) to a stream.

Example:

```
out.write(65); // writes 'A'
```

3. skip()

- Skips a given number of bytes/characters while reading.

Example:

```
in.skip(5); // skips 5 bytes/characters
```

4. close()

- Closes the stream and releases system resources.

Example:

```
in.close();
```

---

18. Write a Java program to copy content of file a.txt to b.txt. (Short & Medium):

```
import java.io.*;

public class CopyFile {
    public static void main(String[] args) throws Exception {
        FileInputStream fin = new FileInputStream("a.txt");
        FileOutputStream fout = new FileOutputStream("b.txt");

        int ch;
        while ((ch = fin.read()) != -1) {
            fout.write(ch);
        }

        fin.close();
        fout.close();

        System.out.println("File copied successfully.");
    }
}
```
